weight-independent in enforcement, Execution Control, Deterministic Logit Masking, and Active Runtime Enforcement in the IDICOC Framework

Ahmad Kamal Salah

Chief Architect & Author of IDICOC

Founder, NewCo

M.Sc. in Information and Communications Systems Management

May 2026

Abstract

In this paper, we introduce the Invariant Data Integrity Chain of Custody (IDICOC), a formal framework designed to enforce bounded state-space drift in autoregressive systems. Addressing the hardware-level precision exhaustion and catastrophic dead-ends inherent in standard constrained decoding frameworks, IDICOC implements a Dual-State Weight-Independent Supervisory Enforcement Controller. By coupling an independent deterministic structural automaton with the inference engine's execution path, the framework introduces single-pass Deterministic Logit Masking paired with an Active Geodesic Fallback insertion protocol. Modeled as a Feed-Forward Active Edit Automaton, IDICOC provides formal proofs of Soundness, strict $O(\log V)$ Bounded Single-Step Resolution, and condition-dependent Information Projection (I-Projection). This establishes a formally bounded theoretical foundation for deterministic AI hardware acceleration without relying on iterative rejection loops or mathematically paradoxical state manipulation.

1. Introduction

Modern autoregressive generative models frequently suffer from latent state drift and semantic hallucination. To mitigate these vulnerabilities, the current state-of-the-art relies on post-hoc constrained decoding (e.g., PICARD, Outlines, LMQL). These frameworks perform late-stage deterministic logit masking prior to token selection.:

However, purely logit-level masking introduces a critical vulnerability when subjected to the physical limits of hardware compute: **Probabilistic Exhaustion**. While the theoretical softmax function never maps to absolute zero, physical inference engines operating in reduced precision formats (e.g., FP16, BF16, INT8) are bound by precision underflow limits. If the underlying model severely hallucinates, the logit values corresponding to the remaining structurally valid tokens may fall below the hardware representation threshold or a defined noise floor (τ), mapping to exactly zero. This leaves a completely nullified functional support space (a catastrophic dead-end).

The IDICOC framework introduces a preventative hardware-level paradigm. Rather than forcing the LLM to sample from exhausted, underflowed distributions, IDICOC implements an independent Dual-State controller. When deterministic logit masking leads to hardware-level probabilistic exhaustion, the framework bypasses the stochastic generator entirely, actively inserting the correct topological token to guarantee structural liveness.

2. The 5-Stage Execution-Coupled Architecture

The operational architecture of the IDICOC framework physically separates the continuous probabilistic generation from the discrete deterministic emission at the execution layer. The state space execution is partitioned into a Cartesian product of five distinct operational manifolds $S = S_0 \times \dots \times S_4$:

- **S_0 (Deviation Quantification Engine - DQE):** The bounded logic evaluation layer. The continuous logit tensor is mapped against the invariant Property Graph G using bounded Hash Look-Up Tables (LUTs).
- **S_1 (Deterministic Logit Masking):** The physical enforcement space where invalid logits are geometrically zeroed via parallel bitwise AND operations.
- **S_2 (Geometric Sandboxing):** The null-routing space for pruned distribution branches.
- **S_3 (Custodial Traceability Module - CTM):** The cryptographic ledger space where verification metadata is sealed.
- **S_4 (Terminal Verified Emission):** The desynchronization-free output layer where the valid token is formally emitted to the environment.

3. Dual-State Tracking, Masking, and Active Insertion

To enforce structural invariants without combinatorial explosion or generative halting, the IDICOC framework relies on tracking the absolute truth, enforcing the mask, and physically overriding the execution path during exhaustion.

3.1 The Independent Syntactic Automaton

The framework maintains an independent, pre-compiled deterministic finite automaton (DFA), denoted as the Property Graph G. The nodes of G represent grammatical or structural states, and the edges represent the token vocabulary Σ.

Crucially, the automaton's active state $q_t \in G$ is determined strictly by the history of *emitted* tokens, possessing perfect independent memory. At any step t, the independent state q_t defines the active mask of permissible next tokens: $E_G(q_t) \subseteq \Sigma$.

The structural nature of the DFA means that IDICOC guarantees structural validity, not semantic omniscience.

3.2 Single-Pass Deterministic Logit Masking

To prevent the emission of invalid states without resorting to iterative rejection sampling, the controller fetches the active subset $E_G(q_t)$ and applies it as a parallel bitwise AND mask across the continuous logit tensor *prior* to token selection. The argmax (or top-p selection) is computed strictly over this truncated distribution.

3.3 Active Geodesic Fallback (Insertion)

When the stochastic generator experiences physical probabilistic exhaustion—meaning the intersection of the hardware-representable predicted support and the valid subset evaluates to the null set—standard masking fails. The IDICOC controller explicitly resolves this by overriding the generator. It bypasses the LM Head sampling array and deterministically inserts the next valid token derived directly from the constraint graph's geodesic path defined by the shortest path metric, to the nearest safe absorbing state, ensuring continuous forward progression.

4. Formal Verification via Feed-Forward Active Edit Automata

We model the IDICOC framework using the theory of **Edit Automata** (Runtime Enforcers), operating independently to control the execution path of the generator.

4.1 The Independent IDICOC Edit Automaton

Let the vocabulary be Σ of size V. An Edit Automaton is a tuple $\mathcal{E} = (Q, q_0, \delta)$, where Q is the independent syntactic state space of the constraint graph G.

Lemma 1 (Graph Completeness — Design Invariant):

The constraint graph G is a pre-compiled state machine over the vocabulary Σ. It is constructed to satisfy the non-emptiness invariant: For any state $q \in Q$, the set of valid outgoing edges $E_G(q)$ is never empty. The Graph Compiler enforces this at compilation time, verifying that an absorbing state w_{safe} is reachable from every state via a finite path.

4.2 Proof of Soundness, Termination, and Liveness

Theorem 1 (Soundness and $O(\log V)$ Bounded Single-Step Resolution):

The IDICOC Edit Automaton guarantees that each individual enforcement step resolves without iterative rejection loops, bounding the step execution strictly by the depth of a logarithmic reduction tree. This ensures strict Noetherian termination per transition, mathematically mapping every emitted state to the codomain of G.

Proof:

While global sequence termination depends on the application-specific graph G, the architectural guarantee relies on strict intra-step circuit complexity. At step t, the valid subset $E_G(q_t)$ is fetched and applied as a parallel bitwise AND mask to the logit tensor. This masking operation exhibits $O(1)$ gate depth.

Subsequently, selecting the token requires either computing the argmax over the masked distribution or triggering the fallback multiplexer. In standard VLSI complexity, computing an argmax over a vocabulary V requires a comparator reduction tree with a physical gate depth of $O(\log V)$ which specifically represents the parallel gate depth, . Because the IDICOC architecture executes the parallel mask and feeds it directly into the deterministic reduction tree or fallback routing—expressly bypassing iterative algorithmic resampling loops—the entire state transition is physically and mathematically bounded by $O(\log V)$ complexity. This bounded, cycle-free physical resolution guarantees strict Noetherian termination per step, structurally ensuring that every emitted state satisfies G.

Definition 1 (Prefix-Closed Liveness):

A language $\mathcal{L}$ is prefix-closed if for every string w in $\mathcal{L}$, every prefix v of w also satisfies $v \in \mathcal{L}$. A generative system exhibits prefix-closed liveness if every valid generated prefix can be extended by a valid transition or safely terminated without deadlocking.

Theorem 2 (Conditional I-Projection and Prefix-Closed Liveness):

The IDICOC framework computes the exact Information Projection (I-Projection) during normative masked generation to ensure step-wise transparency. In events of hardware-level probabilistic exhaustion where I-Projection is mathematically undefined, the framework guarantees prefix-closed liveness via active geodesic insertion.

Proof:

Let the generator's stochastic policy be $\pi(y|h_t)$ over Σ. To model physical execution constraints, we define $\text{supp}_\tau(\pi) \subseteq \Sigma$ as the subset of tokens whose predicted probability exceeds the hardware underflow threshold or noise floor $\tau > 0$. Let the functionally restricted safe vocabulary be $\Sigma_{\text{mask}} = E_G(q_t) \cap \text{supp}_\tau(\pi)$.

The system operates in two mutually exclusive regimes:

Regime A (Normative Masking): When $\Sigma_{\text{mask}} \neq \emptyset$, the hardware yields an enforced distribution $\hat{\pi}(y|h_t)$:

$$\hat{\pi}(y|h_t) = \frac{\pi(y|h_t)}{\sum_{z \in \Sigma_{\text{mask}}} \pi(z|h_t)} \quad \text{if } y \in \Sigma_{\text{mask}}$$

$$\hat{\pi}(y|h_t) = 0 \quad \text{if } y \notin \Sigma_{\text{mask}}$$

In information theory, finding the closest distribution P restricted to Σ_{mask} requires minimizing $D_{KL}(P \parallel \pi)$. This constrained minimization yields the exact **I-Projection** of π onto Σ_{mask}. The unique solution is $\hat{\pi}(y|h_t)$, ensuring step-wise transparency.

Regime B (Hardware Probabilistic Exhaustion): When the latent drift is so severe that all structurally valid tokens fall below τ, the physical intersection evaluates to the null set ($\Sigma_{\text{mask}} = \emptyset$). Consequently, any proposed distribution P restricted to $E_G(q_t)$ places mass strictly outside of $\text{supp}_\tau(\pi)$, rendering $D_{KL}(P \parallel \pi) = \infty$. The

I-Projection is mathematically undefined. In this regime, the IDICOC framework abandons distributional optimization and instead guarantees prefix-closed liveness via deterministic geodesic insertion from $E_G(q_t)$, which, by Lemma 1, is always non-empty.

In both regimes, the emitted token satisfies G. The system seamlessly transitions between optimal transparency (Regime A) and guaranteed structural liveness (Regime B), preventing catastrophic computational deadlocks.

4.3 Theoretical Hardware Complexity and Future Work

To substantiate the architectural viability of the framework, we mathematically decompose the complexity of the enforcement transitions. The IDICOC controller's specific overhead (LUT state lookup coupled with parallel bitwise AND masking) is bounded by strictly $O(1)$ gate depth. The subsequent tensor evaluation (argmax/fallback routing) introduces a latency bounded by $O(\log V)$, which matches the baseline theoretical cost of unconstrained inference. Because all enforcement mechanisms physically bypass iterative loops, the total execution footprint is mathematically deterministic and adds an isolated $O(1)$ asymptotic overhead to standard generation.

While this paper establishes the strict causal bounds and topological viability of the architecture, future empirical research is required. Subsequent phases of this work will focus on formalizing physical Power, Performance, and Area (PPA) metrics on FPGA/ASIC targets, alongside software-level simulations across standardized autoregressive workloads to empirically validate the latency bounds.

5. Conclusion

This paper has formalized the mathematical foundations of the IDICOC framework, systematically resolving the operational contradictions of constrained decoding architectures. By defining IDICOC as a Dual-State Weight-Agnostic Execution Controller, we establish a framework capable of bridging the gap between theoretical state-space constraints and the physical realities of inference engines. Recognizing the computational complexity of algorithmic resampling, IDICOC prevents catastrophic Dead-Ends and Probabilistic Exhaustion through isolated $O(1)$ logit masking paired with Active Geodesic Fallback insertion. Supported by condition-dependent I-Projection optimality and formally proven $O(\log V)$ step resolution, IDICOC establishes a formally bounded safe, deterministic AI hardware execution.